Mizan.ai — Modal Model-Serving Deployment

Issues encountered, fixes applied, and why — a reference for avoiding repeat mistakes. Covers the July 2026 session that took the serving layer from Gemma 3 through Mistral to the final Qwen2.5-based setup.

Contents

1. Quick-reference timeline

Stage	What happened
Gemma 3 → 4	Explored swapping Gemma 3 for Gemma 4; reverted, then removed Gemma entirely (kept brain / translate / embed only).
Brain model swap	Qwen3.5-35B → Mistral-Small-24B-Instruct-2501-AWQ. Added a new lite tier: Qwen2.5-7B-Instruct-GPTQ-Int4.
QwenLite (T4) bugs	Two separate `nvcc`-missing crashes (sampler, then attention backend) — both fixed via config, not code logic changes.
MistralBrain (A10) bugs	Tokenizer crash, then KV-cache OOM, then a silent tool-calling failure traced to a broken chat template.
Model swap #2	Mistral-24B-AWQ → Qwen2.5-14B-Instruct-GPTQ-Int4. Deployed clean on the **first try**, no workarounds needed.
HF Hub stall	A cold-start download hung at 54% for 15+ min. Root cause: anonymous HF Hub rate limiting. Fixed with a real HF_TOKEN secret.
Tool-call parsing	Model was calling tools correctly the whole time; the code just never parsed the output. Implemented and verified a small regex parser.
Final validation	Wrote and ran a test script covering plain chat, Arabic I/O, and multi-turn tool-calling. All passed.

2. Detailed issues, causes & fixes

2.1 — FlashInfer sampler crash: "Could not find nvcc" (QwenLite, T4)

SYMPTOM Engine crashed on first request with `RuntimeError: Could not find nvcc and default cuda_home='/usr/local/cuda' doesn't exist`.

CAUSE vLLM defaults to FlashInfer for top-k/top-p sampling. FlashInfer JIT-compiles a CUDA kernel the first time it's used, which needs the `nvcc` compiler — the Modal image only had `pip install vllm` (runtime CUDA libs), not the full CUDA dev toolkit.

FIX Set `VLLM_USE_FLASHINFER_SAMPLER=0` as an env var on the image (`.env({...})` in the Modal Image definition).

WHY IT WORKS Forces vLLM onto its built-in PyTorch-native sampler, which needs no JIT compilation at all — sidesteps the missing-toolchain problem entirely instead of trying to work around it.

2.2 — Attention backend hit the same nvcc wall (QwenLite, T4)

SYMPTOM Same `Could not find nvcc` error, but deeper in startup (CUDA graph memory profiling), after the sampler fix above.

CAUSE T4's compute capability (7.5) is below FlashAttention2's minimum (8), so vLLM's auto-selection fell through to the FlashInfer *attention* backend — which also JIT-compiles via `nvcc`, independent of the sampler.

FIX, ATTEMPT 1 (FAILED) Set env var `VLLM_ATTENTION_BACKEND=TRITON_ATTN`. Log showed: `WARNING: Unknown vLLM environment variable detected: VLLM_ATTENTION_BACKEND` — this env var was deprecated/removed as of this vLLM version (0.13.0+) in favor of a typed config object.

FIX, ATTEMPT 2 (WORKED) Passed `attention_config=AttentionConfig(backend="TRITON_ATTN")` directly to `LLM()`, importing `AttentionConfig` from `vllm.config` lazily inside the load method.

WHY IT WORKS Triton compiles its own kernels through its own JIT compiler, not `nvcc` — works fine in a toolchain-less image. The env var approach failed not because the idea was wrong, but because the exact mechanism for setting it had moved to a different layer in this vLLM release.

2.3 — Mistral tokenizer crash: `CachedMistralCommonBackend has no attribute is_fast`

SYMPTOM Engine crashed on init: `AttributeError: CachedMistralCommonBackend has no attribute is_fast`.

CAUSE The `stelterlab/Mistral-Small-24B-Instruct-2501-AWQ` repo ships both Mistral's native tokenizer (`tekken.json`) and a standard HF tokenizer (`tokenizer.json`). vLLM auto-detects `tekken.json` and routes to its Mistral-native tokenizer backend — which has a genuine bug in this vLLM version (missing an `is_fast` attribute other code expects).

FIX, ATTEMPT 1 (LOOKED RIGHT, DIDN'T WORK) Passed `tokenizer_mode="hf"` to force the HF backend. Per vLLM's own source logic, this should have bypassed Mistral-native detection entirely (the detection code only triggers when `tokenizer_mode == "auto"`). The log confirmed `tokenizer_mode: 'hf'` was received — yet it crashed into the same Mistral-native backend anyway.

FIX, ATTEMPT 2 (WORKED) Physically deleted `tekken.json` from the downloaded checkpoint directory before vLLM loaded it, in the `load()` method.

WHY THE "OBVIOUSLY CORRECT" FIX FAILED Source-code reasoning about what *should* happen doesn't always match the live behavior of a specific point release — something else in the pipeline was still resolving to the Mistral-native path despite the explicit override. This was never fully root-caused at the vLLM-internals level.

WHY DELETING THE FILE WORKS It removes the physical trigger the auto-detection heuristic keys off — regardless of whatever exact logic decides the tokenizer mode, there's nothing left for it to detect. Stronger than a config flag because it eliminates the failure mode rather than hoping a setting is honored.

2.4 — KV cache out-of-memory on A10 (MistralBrain)

SYMPTOM `ValueError: ... 5.0 GiB KV cache is needed, which is larger than the available KV cache memory (3.03 GiB)`.

CAUSE The 13.3GB AWQ checkpoint plus ~1.4GB of compiled-CUDA-graph overhead left too little of the A10's 24GB to reserve enough KV cache for the model's native 32768-token context window at 90% GPU memory utilization.

FIX Capped `max_model_len=16384` (still 2x the API's 8192 max output-token cap) and raised `gpu_memory_utilization` from 0.9 to 0.95 (safe since Modal dedicates the whole GPU to one container).

WHY IT WORKS KV cache memory requirement scales with `max_model_len` — halving the cap roughly halved the requirement, and the extra utilization headroom added back some of the pool. The fix used the exact numbers vLLM reported in its own error message rather than guessing at a value.

2.5 — Silent logging: our own diagnostics were invisible the entire session

SYMPTOM A debug line added via `logger.info(...)` to inspect the rendered prompt never appeared in Modal's logs, despite the request completing successfully.

CAUSE Python's `logging` module defaults the root logger to `WARNING` level with no handler configured. Nothing in the codebase ever called `logging.basicConfig(level=logging.INFO)`, so *every* `logger.info(...)` call anywhere in the app — including pre-existing ones like "Model already present, skipping download" — had been silently dropped the whole time. Only vLLM's own internally-configured logger (a separate mechanism) and our `logging.exception(...)` ERROR-level calls were ever visible.

FIX Switched the debug line from `logger.info(...)` to plain `print(...)`.

WHY IT WORKS `print()` writes directly to stdout, which Modal captures unconditionally — it bypasses the logging module's level filtering entirely.

LESSON "No output" is not the same as "nothing happened." Verify your own diagnostic is actually capable of appearing before treating its absence as evidence.

2.6 — Mistral tool-calling silently did nothing

SYMPTOM A request with a `calculate_vat` tool schema got back a plain-text clarifying question — the model never attempted to call the tool.

CAUSE Confirmed by printing the actual rendered prompt sent to the model: it contained zero mention of the tool. Then confirmed at the source: read `tokenizer_config.json`'s `chat_template` directly off disk in the running container — this community AWQ repackaging's Jinja template only handles `system` / `user` / `assistant` roles and has no tool-rendering logic at all. Not a vLLM bug — an incomplete artifact of this specific repackaging.

FIX Ultimately resolved by replacing the model entirely (see 2.7) rather than patching this template.

LESSON The earlier "fix" for issue 2.3 (`tokenizer_mode="hf"`, forcing off the Mistral-native tokenizer) had a side effect nobody predicted: it also gave up Mistral's native tool-calling prompt formatting, which only the native path had implemented correctly. Fixing one failure mode silently introduced a different one.

2.7 — Decision: replace Mistral entirely with Qwen2.5-14B-Instruct

DECISION Swapped `MistralBrain` → `QwenBrain` running `Qwen/Qwen2.5-14B-Instruct-GPTQ-Int4`, and removed every Mistral-specific workaround (tekken deletion, `tokenizer_mode` override, `max_model_len` cap, aggressive memory utilization, debug diagnostics).

REASONING Qwen ships a single standard HF tokenizer — no dual-format detection risk at all. Its Hermes-style tool-calling template is one of the most complete and widely-tested in the open-model ecosystem. The 7B sibling (`QwenLite`) was already proven working in this exact environment, making a 14B Qwen model the lower-risk choice on every axis, not just an easier one.

RESULT Deployed clean on the first try with the simplest possible config — no crash, no OOM (9.3GB checkpoint vs. Mistral's 13.3GB left comfortable headroom), no template gap. Confirmed the smaller-surface-area choice was correct.

2.8 — Hugging Face Hub download stalled mid-cold-start

SYMPTOM A checkpoint download froze at 54% (7/13 files) for 15+ minutes with zero progress.

CAUSE Every single cold-start log throughout the entire session carried: `Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN...`. Anonymous requests are rate-limited more aggressively than authenticated ones; repeated redeploys during debugging eventually tipped this from "a bit slower" into a genuine hang.

FIX Created a Modal Secret (`modal secret create huggingface-secret HF_TOKEN=hf_xxx`) and attached it via `secrets=[modal.Secret.from_name("huggingface-secret")]` on all four model classes.

WHY NO OTHER CODE CHANGE WAS NEEDED `ensure_model_on_volume()` already read `token=os.environ.get("HF_TOKEN")` — the secret just needed to actually exist and be attached; the code was already written to use it correctly.

2.9 — Tool-call parsing: the model was right, the code was incomplete

SYMPTOM A request with tools got back `tool_calls: []`, but `content` contained raw `<tool_call>{"name": "calculate_vat", "arguments": {"amount": 1000}}</tool_call>` text.

CAUSE `run_vllm_chat()` had an explicit, pre-existing TODO: it always returned an empty `tool_calls` list, regardless of what the model actually generated. This predates every model swap in this session — it was unfinished scaffolding, not a regression.

FIX Implemented a small regex (`<tool_call>\s*(.*?)\s*</tool_call>`) plus `json.loads()` to extract structured calls from Qwen's Hermes-style output, and strip the matched text back out of `content`. Tested locally against the exact real captured output before deploying.

WHY THIS WAS LOWER-RISK THAN EARLIER FIXES Unlike guessing at vLLM's internal APIs, this only needed to parse the *model's own output text* — a simple, well-documented, stable format, verifiable with a plain local Python script before ever touching the deployed code.

3. Cross-cutting lessons

1 **When "should work per the docs/source" contradicts observed behavior, trust the observation.** `tokenizer_mode="hf"` and `VLLM_ATTENTION_BACKEND` both looked correct on paper and both failed in practice on this specific vLLM point release. Two failed guesses in a row was the signal to stop reasoning from source code and start reading ground truth (actual files, actual logs, actual output) instead.

2 **Verify your diagnostics can actually be seen before trusting their silence.** An entire session's worth of `logger.info()` calls were silently dropped by Python's default logging configuration. "No output" was mistaken for "nothing to report" until it was explicitly checked.

3 **A fix for one failure mode can silently create another.** Routing around the Mistral tokenizer crash (`tokenizer_mode="hf"`) fixed the crash but also quietly broke tool-calling, because both behaviors lived on the same code path. After any workaround, re-test the surrounding functionality, not just the specific symptom that prompted the change.

4 **Prefer removing the trigger over suppressing the symptom.** Deleting `tekken.json` outright was more robust than any config flag, because it eliminated the failure mode's precondition entirely rather than depending on a setting being correctly honored by code we couldn't fully inspect.

5 **When a fragile artifact keeps generating new categories of bugs, stop patching it.** Mistral's community AWQ repackaging produced three unrelated problems in a row (tokenizer crash, OOM, broken template). Switching to a better-supported artifact (official Qwen checkpoints, matching an already-proven sibling model) resolved all three at once, on the first attempt.

6 **Warnings that appear in every single log are worth acting on eventually, not just noting.** The anonymous-HF-Hub-request warning was visible from the very first deploy and was dismissed as background noise until it actually caused a 15-minute stall.

4. Final deployed architecture

Class	Model	GPU	Endpoint	Notes
`QwenBrain`	Qwen2.5-14B-Instruct-GPTQ-Int4	A10	`/generate`	Agent brain, tool-calling confirmed working end-to-end.
`QwenLite`	Qwen2.5-7B-Instruct-GPTQ-Int4	T4	`/generate-lite`	Free-tier chat/RAG, no tool-calling exposed.
`Translator`	MADLAD-400	T4	`/translate`	EN↔AR.
`Embedder`	BGE-M3	T4	`/embed`	1024-dim embeddings.

All four classes authenticate to Hugging Face Hub via the `huggingface-secret` Modal Secret. `QwenBrain` and `QwenLite` both run on the same vLLM version (0.22.1) with no model-specific workarounds remaining in the code.

5. Known open items

- **OPEN** `app/modal_app.py` is a broken duplicate of the real deployment file — it references a `common/` package that doesn't exist next to it, and would fail immediately if actually deployed. It was kept in cosmetic sync throughout this session but never fixed or used for real.
- **OPEN** `app/app.py` and `app/files.zip` are unrelated content (a different "Product Similarity API" project) sitting in the same directory — never addressed.
- **OPEN** `Translator` and `Embedder` were never exercised with real requests in this session — only confirmed via earlier deploy logs, not end-to-end tests like `QwenBrain` got.
- **OPEN** No automated test suite — only the manual smoke-test script (`modal-serving/test_qwen_brain.py`).

Generated from a Claude Code session working on Mizan.ai's Modal serving layer. Source files: `modal-serving/modal_app.py` , `modal-serving/common/model_loader.py` , `modal-serving/test_qwen_brain.py` .